

Tutorial 3 | OS Monsoon'25

Section A

| Author → Rachit Bhandari

1. ELF



What are ELF files?

An ELF → (Executable and Linkable Format) file is a common standard file format used for executables, object code, shared libraries, and core dumps in Unix-like operating systems such as Linux. The ELF format is flexible and extensible, allowing it to be used across different hardware platforms and operating systems.

Why ELF files?

- **Portability** → It's widely used across different hardware architectures and operating systems.
- **Flexibility** → It supports various types of files, including executables, object files, and shared libraries.
- **Modularity** → The use of sections and headers makes it easy to parse and manipulate.

Different Views of an ELF File

Linking View

ELF Header
Program Header Table <i>optional</i>
Section 1
...
Section n
...
...
Section Header Table

Execution View

ELF Header
Program Header Table
Segment 1
Segment 2
...
Section Header Table <i>optional</i>

ELF Header

```

#define EI_NIDENT 16

typedef struct {
    unsigned char    e_ident[EI_NIDENT];
    Elf32_Half      e_type;
    Elf32_Half      e_machine;
    Elf32_Word      e_version;
    Elf32_Addr      e_entry;
    Elf32_Off       e_phoff;
    Elf32_Off       e_shoff;
    Elf32_Word      e_flags;
    Elf32_Half      e_ehsize;
    Elf32_Half      e_phentsize;
    Elf32_Half      e_phnum;
    Elf32_Half      e_shentsize;
    Elf32_Half      e_shnum;
    Elf32_Half      e_shstrndx;
} Elf32_Ehdr;

```

<https://refspecs.linuxfoundation.org/elf/gabi4+/ch4.eheader.html>

ELF Section Header

```

typedef struct {
    Elf32_Word    sh_name;
    Elf32_Word    sh_type;
    Elf32_Word    sh_flags;
    Elf32_Addr    sh_addr;
    Elf32_Off     sh_offset;
    Elf32_Word    sh_size;
    Elf32_Word    sh_link;
    Elf32_Word    sh_info;
    Elf32_Word    sh_addralign;
    Elf32_Word    sh_entsize;
} Elf32_Shdr;

```

<https://refspecs.linuxfoundation.org/elf/gabi4+/ch4.shheader.html>

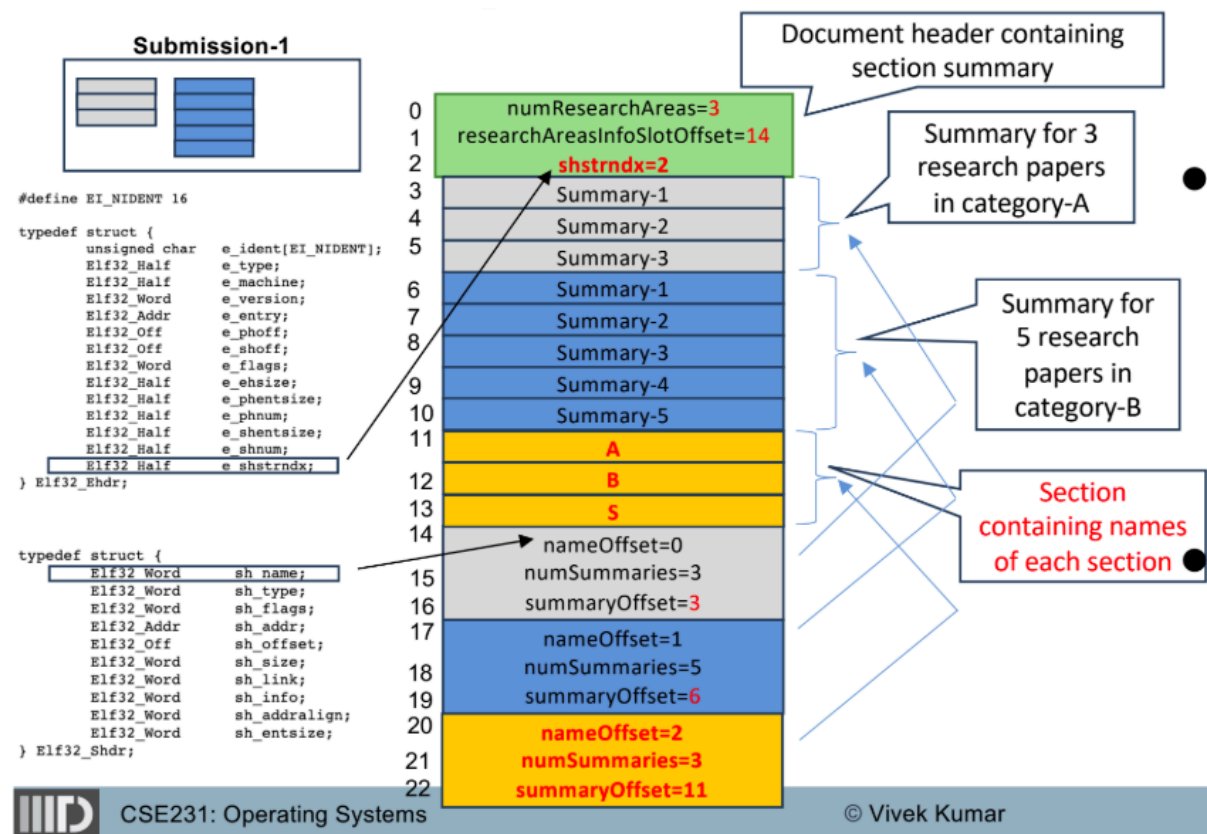
ELF Program Header

```

typedef struct {
    Elf32_Word    p_type;
    Elf32_Off     p_offset;
    Elf32_Addr    p_vaddr;
    Elf32_Addr    p_paddr;
    Elf32_Word    p_filesz;
    Elf32_Word    p_memsz;
    Elf32_Word    p_flags;
    Elf32_Word    p_align;
} Elf32_Phdr;

```

Hands-On



2. File Descriptors and `open()` in C

What is a File Descriptor?

- A **file descriptor (FD)** is a small non-negative integer that represents an open file, socket, pipe, or device in a process.
- It acts as a **handle** (like an index in the kernel's open-file table).
- Every process has its own table of file descriptors.

Standard File Descriptors

FD Number	Symbolic Name	Description
0	STDIN_FILENO	Standard Input (keyboard by default)
1	STDOUT_FILENO	Standard Output (terminal/screen)
2	STDERR_FILENO	Standard Error (terminal/screen for errors)

When you `open()` a file, the **lowest unused FD** is returned.

For example →

- If only stdin, stdout, stderr are open (0, 1, 2 used),
- The next `open()` call returns 3.

The `open()` System Call

Function Prototype

```
#include <fcntl.h>
#include <sys/types.h>
#include <sys/stat.h>

int open(const char *pathname, int flags);
```

Parameters

1. `pathname` (Required)
 - Path to the file (absolute or relative).
 - Example: `"file.txt"` or `"/home/user/data.bin"`
2. `flags` (Required)
 - Defines how the file should be opened.
 - Common values:
 - `O_RDONLY` → Open for read only
 - `O_WRONLY` → Open for write only
 - `O_RDWR` → Open for both read and write

Return Value

- **On success** → returns a **file descriptor** (≥ 0).
- **On failure** → returns `1` and sets `errno`.

Example → Opening Files

```
int fd = open("file.txt", O_RDONLY);
if (fd == -1) {
    perror("open failed");
}
```

Closing File Descriptors

- Once done, **always close** file descriptors to free system resources.

```
close(fd);
```

3. `read()` System Call in C

Me reading instructions
on how to read faster



Me after I realize .. I can't read



The `read()` system call in C is used to **read data from a file descriptor** into a buffer in memory. It is one of the most fundamental low-level I/O operations in Unix/Linux systems.

Function Prototype

```
#include <unistd.h>

ssize_t read(int fd, void *buf, size_t count);
```

Parameters

1. `fd` (`int`)

- File descriptor (obtained from `open()`, etc.).
- It uniquely identifies an open file in the process.
- Examples:
 - `0` → Standard Input (`stdin`)
 - `1` → Standard Output (`stdout`)
 - `2` → Standard Error (`stderr`)
 - `>=3` → User-opened files

2. `buf` (`void *`)

- Pointer to a memory buffer where the data will be stored.
- Must be allocated with enough space to hold at least `count` bytes.

3. `count` (`size_t`)

- Maximum number of bytes to read.
- The actual number of bytes read can be **less than** this value.

Return Value

- `>= 0` (`ssize_t`) → Number of bytes actually read.
 - May be less than `count` if:
 - End of file (`EOF`) is reached.
 - Reading from a terminal or pipe with less data available.
- `0` → End of file (`EOF`).
- `-1` → Error occurred.

Example Usage

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd = open("example.txt", O_RDONLY);
    if (fd < 0) {
        perror("couldn't open the file");
        return 1;
    }

    char buffer[100];
    ssize_t bytesRead = read(fd, buffer, sizeof(buffer) - 1);

    if (bytesRead < 0) {
        perror("couldn't read the file");
        close(fd);
        return 1;
    }

    buffer[bytesRead] = '\0'; // Null-terminate for printing as a string
    printf("Read %zd bytes: %s\n", bytesRead, buffer);

    close(fd);
    return 0;
}
```

Notes

- **Unbuffered** → `read()` works at the kernel level, unlike `fgets()` or `fread()` which provide buffering.
- **Partial reads** → Always check the return value, don't assume you got all the requested bytes.
- **Blocking behavior** → By default, `read()` will block until at least one byte is available (unless the file is non-blocking).

- **EOF detection** → Only when `read()` returns `0`.

4. `lseek()` in C

The `lseek()` system call repositions the file offset of an open file descriptor. It allows random access within files by moving the "cursor" (current read/write position) to a specified location.

Prototype (from `<unistd.h>`) →

```
#include <unistd.h>
#include <sys/types.h>

off_t lseek(int fd, off_t offset, int whence);
```

Parameters

1. `fd` (`int`)

- File descriptor obtained from `open()`.
- Must be a seekable object (regular files usually are, pipes/sockets are not).

2. `offset` (`off_t`)

- The number of bytes to move relative to `whence`.
- Can be positive, zero, or negative (if supported by the file type).

3. `whence` (`int`)

Determines the reference point for the offset:

- `SEEK_SET` → Set the file offset to `offset` bytes from the **beginning** of the file.
- `SEEK_CUR` → Set the file offset to current position + `offset`.
- `SEEK_END` → Set the file offset to file size + `offset`.

Return Value

- On success: returns the new file offset (of type `off_t`).

- On error: returns `1` and sets `errno`.

Examples

1. Move to the beginning of a file

```
int fd = open("data.txt", O_RDONLY);  
lseek(fd, 0, SEEK_SET); // offset now at beginning
```

2. Skip first 100 bytes

```
lseek(fd, 100, SEEK_CUR); // move 100 bytes forward
```

3. Jump to end of file

```
off_t size = lseek(fd, 0, SEEK_END);  
printf("File size = %ld bytes\n", (long) size);
```

4. Negative offset (go back)

```
lseek(fd, -10, SEEK_END); // move to 10 bytes before end
```

Blocking vs Non-blocking?

- Unlike `read()` or `write()`, `lseek()` is **not blocking**.
 - It only adjusts the file offset—it doesn't wait for I/O.
-